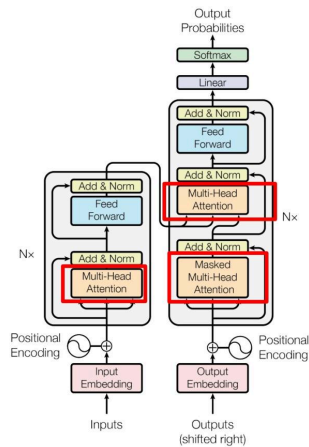
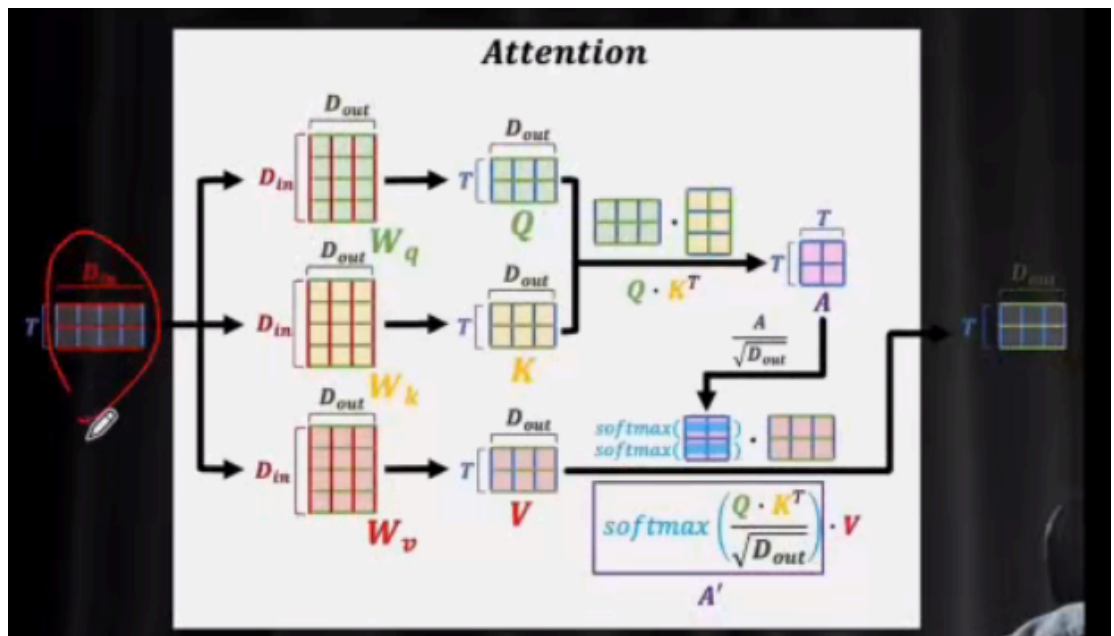
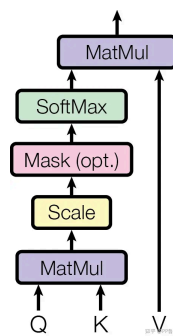


2026/7/30 笔记



Transformer整体架构



注意力机制计算过程

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

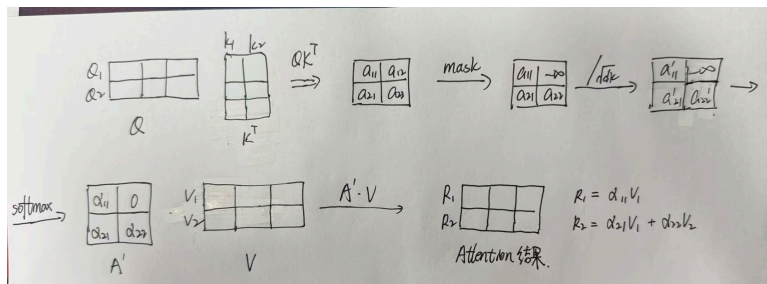
- attention计算过程中的QKV点乘运算主要目的就是：
用于计算Query和Key之间的相关性，通过这个相关性得到注意力权重，然后利用权

重对Value进行加权求和,

使当前位置的词向量融合上下文信息。这个过程不是直接改变QKV的方向,而是在新的语义空间中动态调整

不同词信息的贡献比例,从而生成包含上下文信息的表示。

- 更好理解**加权求和**:



- mask的目的是:

在生成过程中,永远都是根据现有的token预测下一个token,所以当前位置的token只能关注它之前

已经出现的token(包括自己),即使前面的token已经生成了也要按照这种规律来进行计算。

因此mask-attention的结果是一个三角矩阵,其余部分的数值为负无穷,使这部分softmax的结果为0进而

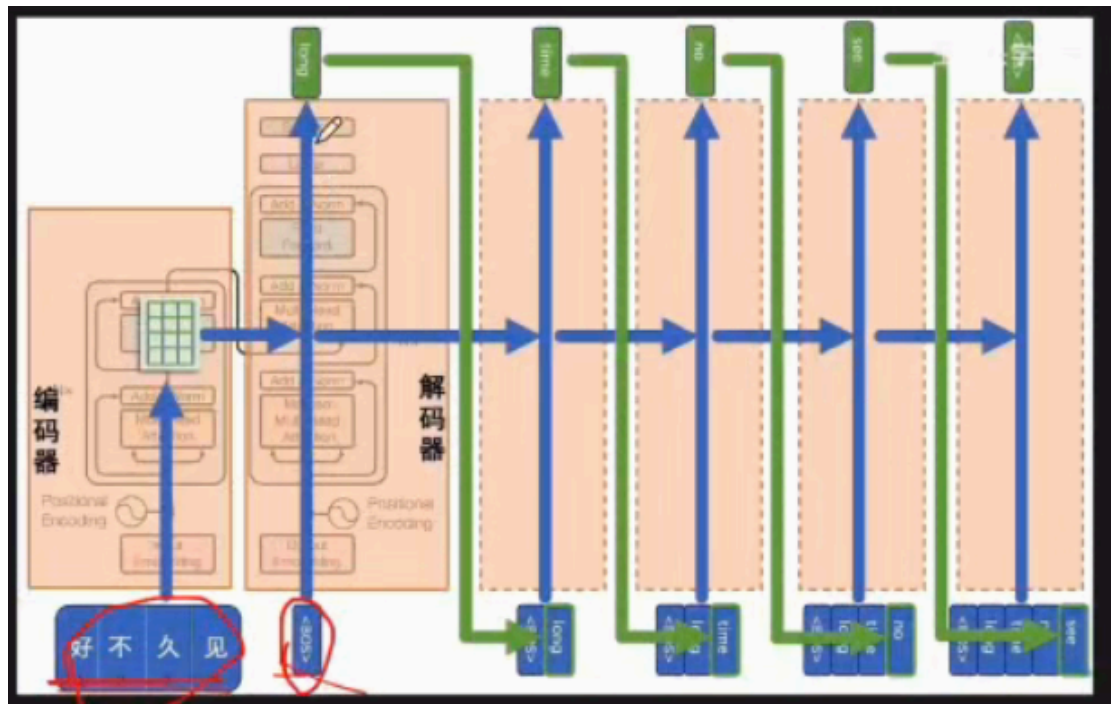
在对V加权的时候,防止当前位置token获取未来token的信息

- 除以 $\sqrt{d_k}$ 的目的是:

向量维数越大,点积的结果也越大,然后进行softmax会导致输出结果很极端,接近one-hot,

进而导致在对于V进行加权的时候只考虑其中一个token。

- 例: $d = 64$, 输入token长度为3, QK^T 结果: $[40, 50, 45]$
则 $\text{Softmax}([40, 50, 45]) \approx [0.00004, 0.993, 0.007]$
- $[40, 50, 45]$ 除以 $\sqrt{64} = 8$ 得到 $[5, 6.25, 5.6]$,
 $\text{Softmax}([5, 6.25, 5.6]) = [0.18, 0.64, 0.18]$



- Encoder-Decoder架构常用于机器翻译：将输入的中文翻译成英文
encoder部分将中文进行嵌入、编码，再经过不同的映射后作为decoder部分的K、V矩阵（这个做法也比较合理，因为是通过V矩阵进行加权来得到下面的翻译结果）。
 - 在解码器部分输入第一个token是 <BOS> 用于表示开始字符
 - 解码器中先进行自注意力计算，然后与编码器结果进行交叉注意力计算
 - 每一步预测出的 Token 会作为下一步 Decoder 的输入：<BOS> -> <BOS>
Long -> <BOS> Long time -> <BOS> Long time no -> <BOS> Long time
no see